

ШИНЭ МОНГОЛ ТЕХНОЛОГИЙН КОЛЛЕЖ
КОМПЬЮТЕРЫН УХААНЫ ИНЖЕНЕРИЙН ТЭНХИМ

Оюутны код: s21c076b
Оюутны овог нэр: Батцэнгэл АНАР

RAG технологид тулгуурласан локал файл
хайлтын ухаалаг туслагч

/ТӨГСӨЛТИЙН СУДАЛГААНЫ АЖИЛ/

Удирдагч багш Н.Соронзонболд
Гүйцэтгэсэн оюутан Б.Анар

Улаанбаатар хот
2025 он

ШИНЭ МОНГОЛ ТЕХНОЛОГИЙН КОЛЛЕЖ
КОМПЬЮТЕРЫН УХААНЫ ИНЖЕНЕРИЙН ТЭНХИМ

Төгсөлтийн судалгааны ажил

RAG технологид тулгуурласан локал файл
хайлтын ухаалаг туслагч

Гүйцэтгэгч: Б.Анар

Удирдагч: Н.Соронзонболд

Улаанбаатар хот
2025 он

Хураангуй

Энэхүү төгсөлтийн судалгааны ажлын зорилго нь интернэт холболтгүй сүлжээгүй орчинд компьютерын дотоод дискэнд хадгалагдсан баримт бичгүүдээс хиймэл оюун ухаанд суурилсан RAG (Retrieval-Augmented Generation) аргаар мэдээлэл хайх систем боловсруулах явдал юм.

Орчин үед мэдээлэл асар хурдтай өсч байгаа боловч байгууллагын дотоод мэдээллийг үр дүнтэй ашиглахад хүндрэл гардаг. Иймд энэхүү судалгаагаар дотоод орчинд ажиллах, өгөгдлийн аюулгүй байдлыг хамгаалсан хиймэл оюун ухааны хайлтын систем боловсруулах замаар уг асуудлыг шийдвэрлэхийг зорьсон.

Төслийн хүрээнд RAG архитектур ашигласан бөгөөд гурван хайлтын аргыг хослуулан ажилладаг: FAISS вектор хайлт (утгачилсан хайлт), BM25 статистик хайлт (түлхүүр үгээр хайлт), мөн шууд тохирол (үгийн тааруулалт). Бичвэрүүдийг Embedding загварт оруулж вектор буюу тоон массив болгон хувиргаж, ONNX INT8 форматаар өгөгдлийг боловсруулахад хурдан болгож CPU дээр хурдан ажиллах боломжийг хангасан. Хэрэглэгчийн интерфэйсийг Gradio сан ашиглан вэб хэлбэрээр хэрэгжүүлсэн бөгөөд Ollama платформоор дамжуулан хэрэглэгчийн асуултанд тулгуурлан холбогдох баримтыг илрүүлэх ба LLM загвараас (Qwen2.5) хариулт үүсгэн авдаг.

Систем нь PDF, DOCX, CSV, XLSX, TXT зэрэг 10 төрлийн файлыг дэмждэг бөгөөд өгөгдлийн нууцлалыг бүрэн хангасан офлайн шийдэл юм.

Түлхүүр үг: RAG, FAISS, BM25, Hybrid хайлт, ONNX, офлайн AI, Ollama, Gradio

Агуулга

Хураангуй	i
Товчилсон үгийн жагсаалт	iii
Хүснэгтийн жагсаалт	iv
Зургийн жагсаалт	v
1 Ажлын төлөвлөгөө	1
2 Удиртгал	2
2.1 Үндэслэл, ач холбогдол	2
2.2 Зорилго, зорилт	2
2.3 Судалгааны хязгаарлалт	3
3 Судалгааны сэдвийн онол, өнөөгийн түвшин	4
3.1 RAG аргачлалын үндэс	4
3.2 Hybrid хайлтын онол	5
3.3 Ижил төрлийн судалгааны ажлуудын харьцуулалт	7
4 Судалгааны арга зүй	8
4.1 Системийн ерөнхий архитектур	8
4.2 Файл уншигч модуль	9
4.3 Chunking (хэсэгчлэл)	10
4.4 Embedding ба ONNX оновчлол	11
4.5 FAISS вектор индекс	11
4.6 Embedding кэш систем	12
4.7 Reranker	12
4.8 LLM хариулт үүсгэгч (Ollama)	12
4.9 Хэрэглэгчийн интерфэйс (Gradio)	13
4.10 Хөгжүүлэлтийн явцад тулгарсан хүндрэлүүд ба шийдлүүд	14
5 Судалгааны үр дүн, дүгнэлт	16
5.1 Хайлтын туршилтын үр дүн	16
5.2 Гүйцэтгэлийн задаргаа (4000 файл)	16
5.3 Оновчлолын үр дүн	17
5.4 Дүгнэлт	17
Ном зүй	18

Товчилсон үгийн жагсаалт

1. **RAG** – Retrieval-Augmented Generation — Хайлтад суурилсан хариулт үүсгэх арга
2. **FAISS** – Facebook AI Similarity Search — Вектор хайлтын сан
3. **BM25** – Best Matching 25 — Статистик түлхүүр үгийн хайлтын алгоритм
4. **ONNX** – Open Neural Network Exchange — Нейрон сүлжээний загвар солилцох стандарт
5. **INT8** – 8-bit Integer Quantization — Загварын хэмжээг багасгах арга
6. **LLM** – Large Language Model — Том хэлний загвар
7. **Embedding** – Бичвэрийг тоон вектор хэлбэрт хөрвүүлэх
8. **UI** – User Interface — Хэрэглэгчийн интерфэйс
9. **PDF** – Portable Document Format — Баримт бичгийн стандарт формат
10. **API** – Application Programming Interface — Програм хооронд харилцах холболт
11. **Chunk** – Баримтыг жижиг хэсгүүдэд хуваасан нэгж
12. **IVF** – Inverted File Index — FAISS-ийн хурдан хайлтын бүтэц
13. **IDF** – Inverse Document Frequency — Баримт дахь үгийн ховор байдлын хэмжүүр
14. **JSON** – JavaScript Object Notation — Өгөгдөл солилцох стандарт бүтэцтэй формат
15. **VSCoде** – Visual Studio Code — Кодыг ажлуулдаг өргөтгөлтэй программ

Хүснэгтийн жагсаалт

1	Судалгааны ажлын төлөвлөгөө	1
2	Дэмжигдэх файлын төрлүүд ба уншигч	9
3	Embedding загварууд	11
4	LLM загваруудын харьцуулалт	12
5	Системийн бодит гүйцэтгэлийн хэмжилт (8 core CPU, 7.7GB RAM)	16
6	Индексэлэлтийн алхам бүрийн хугацаа	16
7	Оновчлолын нөлөө	17

Зургийн жагсаалт

1	RAG-ийн үндсэн бүтэц	4
2	Системийн dataflow диаграмм	8
3	Системийн ерөнхий урсгал (Activation диаграмм)	9
4	Use case диаграмм	13

1 Ажлын төлөвлөгөө

Хүснэгт 1: Судалгааны ажлын төлөвлөгөө

№	Төлөвлөгөө	Эхлэх	Дуусах	Явц
1	Онол, арга зүйн материалуудыг судлах	2025.10.15	2025.10.31	
2	Системийн ерөнхий төлөвлөгөө боловсруулах	2025.11.1	2025.11.15	
3	Диск доторх хайлт болон файлуудыг судлах	2025.11.15	2025.11.30	
4	ТСА үзлэг 1	2025.12.2		
5	Файл уншигч модуль хөгжүүлэх, турших	2025.12.3	2025.12.15	
6	Embedding, FAISS индекс хэрэгжүүлэх	2025.12.15	2025.12.28	
7	BM25 + Keyword хайлт нэмж Hybrid болгох	2026.1.2	2026.1.15	
8	Gradio UI хөгжүүлэх, Ollama холбох	2026.1.15	2026.1.31	
9	ONNX INT8 оновчлол, кэш систем	2026.2.1	2026.2.15	
10	Туршилтын демо бэлтгэх	2026.2.15	2026.2.28	
11	ТСА үзлэг 2	2026.3.1		
12	4000 файлын оновчлол, хурдасгал	2026.3.2	2026.3.15	
13	Хэрэглэгчийн туршилт	2026.3.15	2026.3.31	
14	Судалгааны бичвэрийг боловсруулж дуусгах	2026.4.1	2026.4.15	
15	Туршилтын үр дүнг нэгтгэх	2026.4.15	2026.4.29	
16	ТСА урьдчилсан хамгаалалт	2026.4.30		
17	Сайжруулалт хийх	2026.5.1	2026.5.15	
18	ТСА хамгаалалтанд бэлэн болох	2026.5.15	2026.5.30	
19	ТСА үндсэн хамгаалалт	2026.5.31		

2 Удиртгал

2.1 Үндэслэл, ач холбогдол

Орчин үед мэдээллийн хэмжээ хурдацтай өсч, байгууллага болон хувь хүний компьютерт олон төрлийн их хэмжээний өгөгдлийг богино хугацаанд боловсруулах, хайх, дүн шинжилгээ хийх шаардлага улам нэмэгдэж байна. Гэвч компьютерын дотоод дискэнд хадгалагдсан файлуудаас шаардлагатай мэдээллийг гараар хайх нь цаг их шаарддаг, алдаа гарах магадлал өндөр үйл явц юм.

APQC-ийн 2023 оны KM Benchmark судалгаагаар мэдлэгийн ажилтнууд ажлынхаа 20–30%-ийг зөвхөн мэдээлэл хайх эсвэл дахин бүтээхэд зарцуулдаг бөгөөд энэ нь өдөрт дунджаар 1.8–2 цаг орчим юм. Хайлтын систем сайн ажилладаг байгууллагуудад энэ хугацааг 50% хүртэл багасгаж чаддаг гэж тооцдог.

Мөн ихэнх RAG системүүд (ChatGPT, Perplexity гэх мэт) нь интернэт холболт шаарддаг тул байгууллагын нууц мэдээллийг гадаад серверт илгээх гэх мэт аюулгүй байдлын хувьд эрсдэлтэй. Иймээс бүрэн офлайн орчинд ажилладаг, өгөгдлийн нууцлалыг хамгаалсан RAG систем боловсруулах шаардлагатай гэж үзсэн.

2.2 Зорилго, зорилт

Энэ судалгааны зорилго нь компьютерын дотоод дискэнд хадгалагдсан бичвэрийн өгөгдлийг боловсруулан, хэррэглэгчийн асуултанд тохирох мэдээллийг автоматаар илрүүлж, холбогдох хариултыг гаргаж чаддаг RAG (Retrieval-Augmented Generation) технологид суурилсан хариултын системийг боловсруулан ажлын цаг хэмнэх, үр бүтээмжтэй байдлыг нэмэгдүүлэх юм.

Зорилт:

- Локал дискнээс бичвэрийн төрлийн файл уншиж, вектор хэлбэрт хөрвүүлэх процессыг хэрэгжүүлэх
- FAISS вектор хайлт (утгачилсан хайлт), BM25 статистик хайлт (түлхүүр үгээр хайлт), мөн keyword matching (үгийн шууд тааруулалт) гэсэн гурван аргыг нэгтгэсэн Hybrid хайлт бүтээх
- Хэрэглэгчийн асуултанд үндэслэн утгачилсан хариулт үүсгэх LLM загвар ажлуулах
- ONNX INT8 оновчлол ашиглан индекс үүсгэх болон хайлтын процессыг CPU дээр хурдан ажиллах боломжтой болгох
- Хайлтын системийг туршиж оновчтой байдлыг нэмэгдүүлэх
- Gradio ашиглан хэрэглэгчид хялбар вэб интерфэйс хэрэгжүүлэх

2.3 Судалгааны хязгаарлалт

Систем нь дотоод орчинд (сүлжээгүй орчинд) бүрэн ажилладаг. Гэвч дараах хязгаарлалттай:

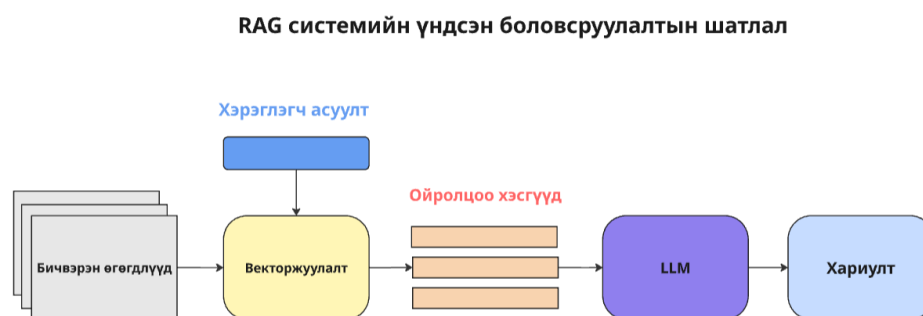
- Зөвхөн бичвэрт суурилсан файлуудыг дэмждэг — зураг, аудио, видео файл уншиж чадахгүй
- Зурган PDF (scan хийсэн) файлаас текст гаргахад OCR буюу тэмдэгт таних оптик технологийн шаардлагатай
- Локал LLM загварын хариултын чанар GPT-4 зэрэг том загвараас доогуур мөн англи хэлнээс бусад хэл дээр нарийвчлал багатай
- Монгол хэлний вектор загварын нарийвчлал хязгаарлагдмал (multilingual загвар ашиглаж байгаа)
- GPU байхгүй тохиолдолд вектор хийх хурд удаан байж болох бөгөөд энэ нь индекс үүсгэх болон хайлтын хурданд нөлөөлнө. Энэ системд CPU дээр тулгуурлан индекс үүсгэж байгаа тул хурд удаан байж болохыг анхаарах хэрэгтэй.

3 Судалгааны сэдвийн онол, өнөөгийн түвшин

3.1 RAG аргачлалын үндэс

RAG (Retrieval-Augmented Generation) нь хайлт (retrieval) болон хэлний загварын үүсгэлт (generation)-ийг нэгтгэдэг аргачлал юм. Уламжлалт LLM загварууд зөвхөн сургалтын үеийн мэдлэгт тулгуурладаг бол RAG нь гадаад эх сурвалжаас мэдээлэл татан авч загварт контекст болгон өгдгөөрөө давуу талтай. Энэхүү зарчмыг “*нэвтрэлт* → *хайлт* → *үүсгэлт*” гэсэн гурван үе шатаар тайлбарлаж болно.

Зураг 1-т RAG-ийн үндсэн бүтцийг харуулав.



Зураг 1: RAG-ийн үндсэн бүтэц

3.1.1 Retriever бүрэлдэхүүн

Retriever нь RAG-ийн “хайлт” хэсгийг хариуцдаг. Баримтыг векторт хөрвүүлж хадгалах ба асуулт ирэхэд хамгийн холбогдох хэсгүүдийг тодорхой алгоритмаар олдог. Retriever-ийн гол зарчим нь:

1. **Indexing үе** — Бичвэрүүдийг embedding загвараар тоон вектор болгон FAISS индексэд хадгална.
2. **Query боловсруулах үе** — Хэрэглэгчийн асуултыг мөн адил вектор болгон хөрвүүлнэ.
3. **Хайх үе** — Хоёр вектор хоорондын косинус зайгаар хамгийн ойрхон утгатай баримтыг олдог.

Энэ системд Retriever-ийн доор гурван давхар дэд бүрэлдэхүүн оршдог — эдгээрийг §3.2-т нарийвчлан тайлбарлана.

3.1.2 Generator (LLM) бүрэлдэхүүн

Generator нь Retriever-аас ирсэн контекст өгөгдөл дээр үндэслэн хүнд ойлгогдохуйц хариулт үүсгэдэг хэлний загвар юм. RAG технологид Generator нь шинэ мэдлэгийг LLM-ийн сургагдсан мэдлэгээс биш, харин Retriever-ийн өгсөн баримтаас уншдаг — энэ нь hallucination (буруу мэдээлэл зохиох) аюулыг эрс буурулдаг давуу тал юм.

Энэ системд Generator-ийн үүргийг Ollama платформ дээр ажиллах Qwen2.5 загвар гүйцэтгэдэг бөгөөд энэ талаар §4.8-т дэлгэрэнгүй тайлбарлана.

3.1.3 Ажиллах ерөнхий дараалал

1. Баримт бичгүүдийг уншиж, chunk хэлбэрт хуваана
2. Chunk бүрийг embedding загвараар тоон вектор болгон FAISS-д хадгална
3. BM25 статистик индекс бүтээнэ
4. Хэрэглэгч асуулт оруулахад гурван аргаар зэрэг хайна
5. Хамгийн холбоотой Top-K chunk(ойролцоо мэдээлэл)-ийг LLM-д контекст болгон өгнө
6. LLM хариулт үүсгэж хэрэглэгчид харуулна

3.2 Hybrid хайлтын онол

Нэг хайлтын аргад найдах нь ерөнхий нөхцөлд хамгийн сайн үр дүн өгдөггүй: вектор хайлт утгачилсан асуултанд давуу ч яг нэг үгийн хайлтанд дутмаг, харин BM25 нь эсрэг зарчмаар ажилладаг. Иймд энэ систем гурван аргыг хослуулсан **Hybrid** буюу олон талт шийдлийг ашигласан.

3.2.1 Вектор хайлт (FAISS)

FAISS сан нь бичвэрийн утга, агуулгыг 384 хэмжээст тоон вектор болгон хувиргаж, косинусын томъёогоор векторуудын хоорондын зайг тооцоолон хамгийн ойролцоо утгатай баримтыг олдог. Үүний давуу тал нь ижил санааг өөр үгээр илэрхийлсэн баримтыг ч илрүүлэх боломжтой байдаг. Жишээлбэл, хэрэглэгч “ухаалаг гэрэлтүүлгийн систем” гэж хайлт хийхэд “автомат гэрэл удирдлагын технологи” гэсэн агуулгатай баримтыг мөн олж чадна. FAISS индексийн хоёр горим:

- **IndexFlatIP** — Бүх векторуудыг шууд харьцуулж, косинус төстэй байдлыг тооцоолон хамгийн ойролцоо утгатай хэсгийг олдог энгийн хайлтын арга. Өгөгдлийн хэмжээ бага үед (жишээ нь $chunk \leq 5000$) илүү тохиромжтой бөгөөд хайлт нь нарийвчлал өндөртэй байдаг.

- **IndexIVFFlat** — Векторуудыг эхлээд бүлэг (кластер) болгон хувааж, хайлт хийх үед зөвхөн хамгийн боломжит бүлгүүд дотор хайлт хийдэг арга. Энэ нь өгөгдлийн хэмжээ их үед (жишээ нь $\text{chunk} > 5000$) хайлтыг илүү хурдан болгох давуу талтай.

3.2.2 Статистик хайлт (BM25)

BM25 (Best Matching 25) нь текст хайлтад ашиглагддаг алгоритм бөгөөд баримт бүрийг хэрэглэгчийн хайсан үгтэй хэр зэрэг тохирч байгаагаар нь оноогоор эрэмбэлдэг. Энэ алгоритм нь хоёр үндсэн хүчин зүйлд тулгуурладаг. Нэгдүгээрт, үгийн давтамж буюу хайсан үг тухайн баримт дотор хэдэн удаа давтагдсаныг тооцоолдог. Хэрэв тухайн үг олон удаа гарвал баримтын оноо нэмэгдэнэ. Хоёрдугаарт, баримтын урт буюу текстийн нийт үгийн тоог харгалзан үздэг. Урт баримтад үг олон удаа таарах магадлал өндөр байдаг тул алгоритм үүнийг жингээр тохируулж, хэт урт баримт давуу байдал авахгүй байхаар тооцоолол хийдэг. Ийм байдлаар үгийн давтамж болон баримтын уртыг хамтад нь тооцоолж, хэрэглэгчийн хайсан үг тухайн үгийг агуулсан баримтыг оноогоор эрэмбэлэн олдог. Тиймээс тодорхой түлхүүр үг дээр суурилсан хайлтад BM25 нь ихэвчлэн вектор хайлтаас илүү нарийвчлалтай ажиллах боломжтой. BM25 оноог дараах томъёогоор тооцдог:

$$\text{BM25}(q, d) = \sum_{t \in q} \underbrace{\log \left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1 \right)}_{\text{IDF}} \cdot \underbrace{\frac{f_{t,d} (k_1 + 1)}{f_{t,d} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)}}_{\text{TF нормчлол}}$$

Энд N нийт баримтын тоо, n_t тухайн үг агуулсан баримтын тоо, $f_{t,d}$ баримт d дахь t үгийн давтамж, $k_1 = 1.5$, $b = 0.75$ нь тогтмол параметрууд юм.

3.2.3 Шууд тэмдэгтийн хайлт (Keyword Matching)

Шууд тэмдэгтийн хайлт нь текст доторх үг, тэмдэгтүүдийг яг ижил дарааллаар нь харьцуулж таарч байгаа эсэхийг шалгах энгийн боловч үр дүнтэй арга юм. Энэ арга нь баримтын текстийг индексжүүлсний дараа хэрэглэгчийн оруулсан түлхүүр үгийг тухайн индекс дотор шууд хайж, яг ижил тэмдэгтийн дараалал таарах хэсгийг олдог.

Ийм төрлийн хайлт нь нэр, код, дугаар, тусгай нэр томъёо зэрэг яг ижил хэлбэрээр бичигдсэн мэдээллийг олоход илүү тохиромжтой. Учир нь вектор хайлт нь утгын ойролцоо байдлыг, BM25 алгоритм нь үгийн давтамжийг харгалзан баримтыг эрэмбэлдэг бол шууд тэмдэгтийн хайлт нь текст доторх тэмдэгтийн дарааллыг шууд харьцуулж шалгадаг. Иймээс “CS101”, “AB-203”, “Node-RED” зэрэг нарийн код, тусгай тэмдэгт агуулсан нэршлийг илрүүлэхэд энэ арга илүү найдвартай ажилладаг.

3.2.4 Гурван аргын эцсийн нэгтгэл

Гурван аргын оноог нэгдсэн нийлбэрээр эцсийн оноо болгон тооцно:

$$\text{Score}_{\text{total}} = 0.45 \times V_{\text{score}} + 0.40 \times B_{\text{score}} + 0.15 \times K_{\text{score}}$$

Энд V нь FAISS вектор оноо, B нь BM25 оноо, K нь түлхүүр үгийн оноо юм. Жингийн харьцааг хайлтын зорилгоос хамааран хэд хэдэн өөр тохиргоогоор (хайлтын горим) ашигладаг. Үүнд: *factual* (баримтад суурилсан хайлт), *semantic* (утга, санааны ойролцоо хайлт), *exact* (яг ижил түлхүүр үг таарах хайлт), *mixed* (дээрх аргуудыг хослуулсан хайлт) зэрэг багтана. Эдгээр хайлтын горимууд нь BM25, вектор хайлт болон түлхүүр үгийн хайлтын үр дүнд өгөх жинг өөр өөрөөр тохируулж, тухайн хэрэглээний шаардлагад тохирсон үр дүнг гаргахад ашиглагддаг.

3.3 Ижил төрлийн судалгааны ажлуудын харьцуулалт

1. **Lewis et al. (2020)** — RAG-ийн анхны судалгаа. DPR + BART загвараар Wikipedia-аас хариулт үүсгэсэн [1]. Гэвч бүх мэдээлэл онлайн Wikipedia-д суурилсан тул локал файлд хэрэглэх боломжгүй.
2. **LangChain RetrievalQA** — LangChain framework-ийн бэлэн chain ашиглан баримт бичгээс хариулт үүсгэх [5]. OpenAI API шаарддаг, офлайн орчинд ажиллуулахад нэмэлт тохиргоо хэрэгтэй.
3. **PrivateGPT** — Бүрэн локал орчинд ажилладаг RAG систем. LlamaCPP + Qdrant ашигладаг [12]. Гэхдээ зөвхөн PDF, TXT дэмждэг бөгөөд Hybrid хайлт (BM25+Vector) хэрэгжүүлээгүй.
4. **Haystack (deepset)** — Модульчлагдсан NLP framework. ElasticSearch/FAISS retriever ашигладаг [11]. Серверт суурилсан архитектур тул хэрэглэгчийн компьютерт ажиллуулахад хүнд.
5. **Benedetti (2024/2025)** — GPT-3.5/4 ашиглан open-domain QA-д зориулсан RAG системийг RAGAS framework-оор үнэлсэн [8]. API-based тул офлайн ашиглах боломжгүй.

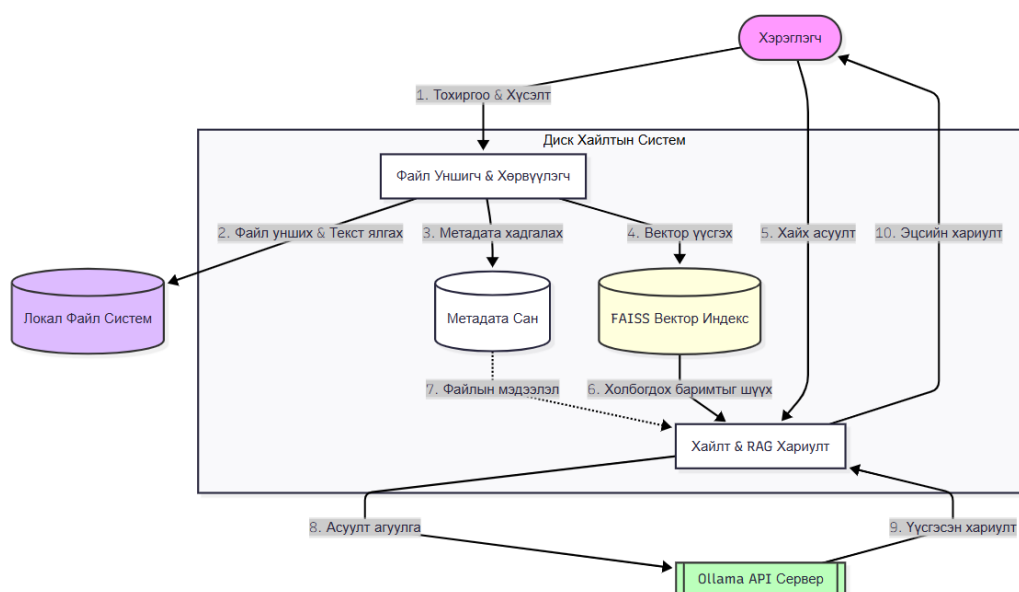
Дээрх харьцуулалтаас харахад ихэнх RAG систем нь нэг хайлтын арга, онлайн API, эсвэл цөөн файлын формат дэмждэг. Энэ систем нь **Hybrid хайлт (3 арга)**, **бүрэн сүлжээгүй орчинд**, **10 файлын формат унших**, **ONNX оновчлол буюу бага тооцоолол** хослуулснаараа ялгаатай.

4 Судалгааны арга зүй

Энэхүү судалгаа нь хэрэглээний чиглэлийн судалгаа бөгөөд энэ нь хүмүүсийн хувийн болон компанийн файл, бичиг баримттай харьцах, хайх хугацааг багасган илүү үр дүнтэй хурдан ажиллахад хэрэг болоход чиглэсэн. Чанарын судалгаа ашиглан ямар хиймэл оюун ухааны модел илүү хариулт гаргаж байгааг үнэлсэн.

4.1 Системийн ерөнхий архитектур

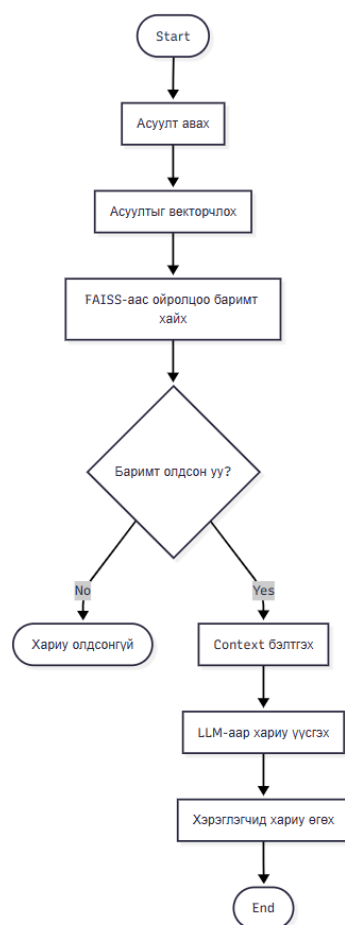
Системийн нийт урсгалыг дараах схемд харуулав:



Зураг 2: Системийн dataflow диаграмм

Систем нь гурван гол давхаргаас бүрдэнэ:

- **Оролтын давхарга** — Локал дискнээс файл унших, бичвэрт хөрвүүлэх, chunk болгох.
- **Индексийн давхарга** — FAISS вектор индекс, BM25 статистик индекс, embedding кэш.
- **Хариултын давхарга** — Hybrid хайлт, Reranker, Ollama LLM, Gradio UI.



Зураг 3: Системийн ерөнхий урсгал (Activation диаграмм)

4.2 Файл уншигч модуль

4.2.1 Дэмжигдэх форматууд

Систем нь дараах файлын төрлүүдийг уншиж, бичвэрт хөрвүүлдэг:

Хүснэгт 2: Дэмжигдэх файлын төрлүүд ба уншигч

Өргөтгөл	Тайлбар	Ашигласан сан
.pdf	Баримт бичиг	PyMuPDF / pdfplumber / pypdf (3 шатлал)
.docx	Word баримт	python-docx
.txt, .md, .log, .rst	Текст файлууд	Built-in (4 encoding)
.csv, .tsv	Хүснэгт	pandas
.xlsx, .xls	Excel хүснэгт	pandas (openpyxl)

4.2.2 PDF-ийн 4 шатлалт fallback механизм

PDF уншихад хамгийн их хүндрэл гардаг тул 4 шатлалт алдаа барих механизм хэрэгжүүлсэн:

1. **PyMuPDF (fitz)** — Хамгийн хурдан, ихэнх PDF-д сайн ажиллана. `fitz.open()` функцийг ашиглан хуудас бүрийн текстийг `get_text("text")` аргаар авна.
2. **pdfplumber** — Япон болон CJK фонтойд PDF-д илүү сайн. Stage 1 амжилтгүй болсон тохиолдолд автоматаар шилжинэ.
3. **pypdf** — Хамгийн өргөн нийцтэй. PDF файлын толгойг `b"%PDF"` гэж шалгасны дараа хуудас бүрийн текстийг `extract_text()` аргаар авна.
4. **Placeholder** — Гурван stage бүгд амжилтгүй болсон тохиолдолд файлын нэрийг индексэд оруулж, “Зурган PDF” гэсэн тэмдэглэл үлдээнэ. Ингэснээр файл хэзээ ч чимээгүй алдагддаггүй.

4.2.3 Шүүлтийн механизм

Зөв баримт уншихын тулд систем дараах шүүлтийг хэрэгжүүлдэг:

- **IGNORE_DIRS** — `.git`, `venv`, `node_modules`, `__pycache__` зэрэг 40+ техникийн директорыг алгасдаг.
- **SKIP_EXTENSIONS** — `.py`, `.js`, `.png` зэрэг кодын болон медиа файлуудыг алгасдаг.
- **MAX_FILE_MB** — 50MB-аас том файлыг алгасдаг.
- **MIN_CHUNK_LEN** — 10 тэмдэгтээс богино мөрийг chunk болгохгүй.

4.3 Chunking (хэсэгчлэл)

4.3.1 Ерөнхий зарчим

Том файлуудыг жижиг хэсгүүдэд хуваах нь RAG системийн хайлтын нарийвчлалд шийдвэрлэх нөлөөтэй. Хэт том chunk нь холбогдохгүй мэдээллийг агуулж, хэт жижиг chunk нь утга алддаг.

4.3.2 Тохиргоо ба параметрууд

- **Chunk хэмжээ:** 900 тэмдэгт — хэд хэдэн туршилтын дараа оновчилсон утга.
- **Давхцал (overlap):** 120 тэмдэгт — хэсгүүдийн хил дээрх мэдээлэл алдагдахаас хамгаална.
- **Хуваах цэг:** “.”, “\n”, “\n\n” зэрэг байгалийн тасалдлаар хуваана.

4.3.3 Файл болон chunk хоорондын харилцаа

Нэг файл олон chunk-т хуваагдах бөгөөд chunk тус бүрт эх файлын нэр болон зам хадгалагдана. Энэ нь хариулт үүсгэх үед эх сурвалжийн мэдээллийг дурдах боломжийг олгодог.

4.4 Embedding ба ONNX оновчлол

4.4.1 Embedding ерөнхий ойлголт

Embedding гэдэг нь бичвэрийн утгыг тоон вектор хэлбэрт хөрвүүлэх үйл явц юм. Ойролцоо утгатай өгүүлбэрүүд тоон орон зайд ойрхон байрлах ба холын утгатай нь холдоно. Энэ шинж чанар нь утгачилсан хайлтын суурь юм.

4.4.2 Загварын сонголт

Хүснэгт 3: Embedding загварууд

Нэр	Хэмжээ	Монгол	Хурд
multilingual-fast	470MB	Тийм	Дундаж
multilingual-quality	470MB	Тийм	Удаан
ultra-fast	17MB	Үгүй	Маш хурдан
fast	80MB	Үгүй	Хурдан

4.4.3 ONNX INT8 quantization

ONNX (Open Neural Network Exchange) форматт шилжүүлж, INT8 quantization хийснээр:

- Загварын хэмжээ 50% буурдаг
- CPU дээрх тооцооллын хурд 2–3 дахин нэмэгддэг
- Intel OpenVINO нэмбэл дахин 3–5 дахин хурдасдаг

Энэ оновчлолыг `ORTModelForFeatureExtraction` болон `quantize_dynamic()` функцуудаар хэрэгжүүлсэн.

4.5 FAISS вектор индекс

4.5.1 Ерөнхий зарчим

FAISS нь Facebook-ийн хөгжүүлсэн вектор хайлтын сан бөгөөд хэдэн сая вектор дундаас хамгийн ойрхон хөршийг хурдан олох зориулалттай. Систем нь chunk тоо 5000-аас доош бол шууд `IndexFlatIP`, их бол `IndexIVFFlat` ашигладаг.

4.5.2 IVF бүтэц

`IndexIVFFlat` нь векторуудыг $\sqrt{n}$ кластерт хуваадаг. Хайлтад бүх векторыг шалгахын оронд зөвхөн хамгийн ойр кластеруудыг шалгаснаар хурд эрс нэмэгддэг. `nprobe` параметр нь шалгах кластерийн тоог тодорхойлно (6–24).

4.6 Embedding кэш систем

Системийг дахин ажиллуулах бүрт бүх файлыг дахин embedding хийх нь цаг их зарцуулдаг байсан. Энэ асуудлыг шийдэхийн тулд MD5 hash-д суурилсан кэш систем хэрэгжүүлсэн:

1. Chunk бүрийн MD5 hash-ийг тооцоолно.
2. Кэшт байвал хадгалагдсан embedding-ийг шууд ашиглана.
3. Зөвхөн шинэ эсвэл өөрчлөгдсөн chunk-ийг дахин тооцоолно.
4. `FileTracker` класс нь файлын hash болон өөрчлөгдсөн хугацааг хадгалдаг.

Энэ оновчлолын ачаар дахин индексэлэх хугацаа 614 секундээс 5 секунд болж буурсан.

4.7 Reranker

Эхний хайлтын үр дүнг дахин эрэмбэлж нарийвчлалыг сайжруулдаг. Cross-Encoder загварыг (`ms-marco-MiniLM-L-6-v2`) ашигладаг бөгөөд асуулт болон chunk-ийг хосоор нь оролт болгон оноо гаргадаг. Ялангуяа Англи хэлний асуултад мэдэгдэхүйц сайжруулалт өгдөг.

4.8 LLM хариулт үүсгэгч (Ollama)

4.8.1 Ollama платформ

Ollama нь LLM загваруудыг локал орчинд ажиллуулах зориулалттай платформ юм. Интернет шаардахгүй бөгөөд `http://localhost:11434` хаягаар REST API-аар харилцдаг.

4.8.2 Загварын сонголт

Туршилтын үр дүнгээс (Хүснэгт 4) харвал олон хэлийг дэмждэг, хөнгөн Qwen2.5 загварыг сонгосон. Загвар нь 0.5B-аас 7B хүртэл параметртэй хувилбартай бөгөөд CPU-тай тохиромжтой байдлаараа давуу болсон.

Хүснэгт 4: LLM загваруудын харьцуулалт

4.8.3 Prompt бүтэц

LLM-д дамжуулах prompt нь гурван хэсгээс бүрдэнэ:

1. **Системийн заавар** — Хариулах хэл, эх сурвалж дурдах дүрэм.
2. **Контекст** — Retriever-аас олдсон Top-K chunk-ийн бичвэр.
3. **Хэрэглэгчийн асуулт** — Шууд асуулт.

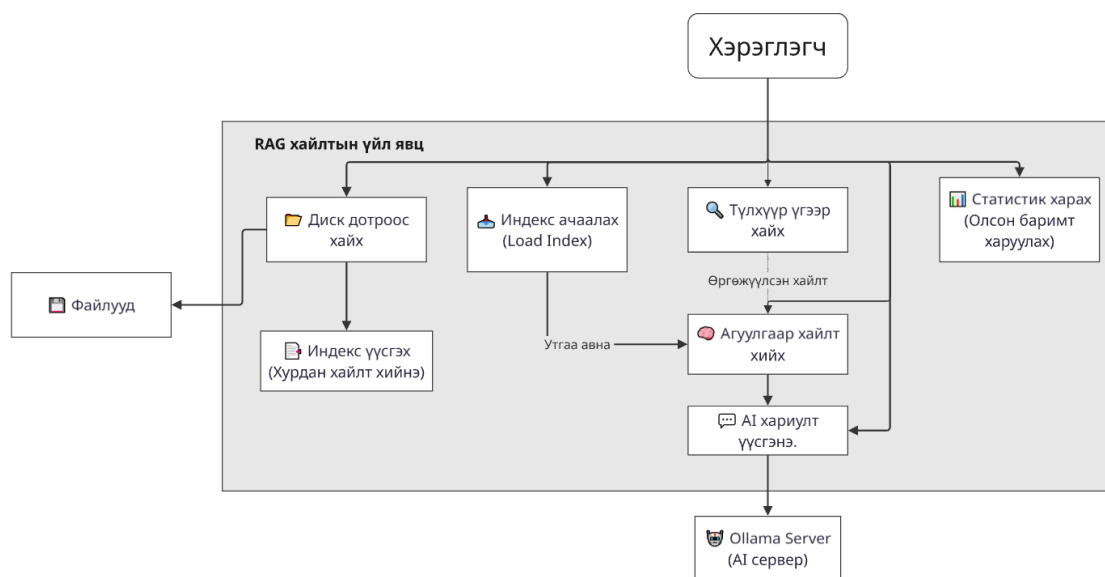
Streaming горимоор хариулт үг үгээр гардаг тул хэрэглэгч урт хариулт хүлээхгүй.

4.9 Хэрэглэгчийн интерфейс (Gradio)

4.9.1 Gradio сангийн тухай

Gradio нь Python кодоор хурдан вэб интерфейс бүтээх сан юм. Систем нь `localhost:7860` хаягаар хөтөч дээр нээгддэг бөгөөд интернэт шаардахгүй.

4.9.2 Интерфейсийн бүтэц



Зураг 4: Use case диаграмм

Интерфейс нь хоёр таб агуулдаг:

- **Асуулт таб** — Чат хэлбэрээр асуулт оруулж, хариулт харах. Debug горимд хайлтын оноо болон эх сурвалжийг харуулна.
- **Файл шалгах таб** — Тодорхой файл индексэд байгаа эсэх, хэдэн оноо авч байгааг оношилдог.

4.9.3 Индекс удирдлага

Хэрэглэгч нэг удаа индексэлсний дараа хадгалагдсан индексийг хэзээ ч ачаалж ашиглах боломжтой. Индекс нь `.faiss` болон `.meta` хоёр файлд хадгалагддаг.

4.10 Хөгжүүлэлтийн явцад тулгарсан хүндрэлүүд ба шийдлүүд

Системийг хөгжүүлж турших явцад зөвхөн кодын синтаксийн алдаа биш, харин өгөгдөл боловсруулах арга болон системийн ажиллагааны урсгалтай холбоотой хэд хэдэн бодит хүндрэл гарсан. Доор хамгийн их тулгарсан асуудлуудыг тус бүрийн шийдлийн хамт тайлбарлав.

4.10.1 Chunking хэмжээ хангалтгүйгээс хайлтын үр дүн буурах

Асуудал: Эхний хувилбар дээр том хэмжээтэй файлыг бүхэлд нь нэг embedding болгон хадгалж байсан. Баримтын тодорхой хэсэгт байрласан мэдээллийг хайлтаар олоход хүндрэл гарсан.

Шийдэл: Chunk хэмжээг 600-аас 900 тэмдэгт болгон нэмэгдүүлж, 120 тэмдэгтийн давхцал (overlap) нэмснээр хайлтын нарийвчлал мэдэгдэхүйц сайжирсан. Нийт chunk тоо 90,000-аас 60,862 болж буурч, боловсруулалтын хурд нэмэгдсэн.

4.10.2 FAISS индексийн хэмжээ зөрөх алдаа

Асуудал: Embedding загварыг өөрчлөх үед өмнө үүсгэсэн FAISS индексийг ашиглахад “dimension mismatch” алдаа гарсан. Шинэ embedding-ийн хэмжээс хуучин индексийн хэмжээсээс өөр болсонтой холбоотой.

Шийдэл: Индексийг ачаалах үед хэмжээсийг шалгаж, зөрүү илэрвэл хуучин индексийг устган шинээр үүсгэдэг хамгаалалт нэмсэн.

4.10.3 PDF файлаас текст унших хүндрэл

Асуудал: Зарим PDF файлаас текст бүрэн гарч ирэхгүй, эсвэл унших хугацаа удаашрах асуудал гарсан. Ялангуяа scan хэлбэрийн буюу зураг хэлбэртэй PDF дээр текст хэлбэрийн өгөгдлийг шууд олох боломжгүй байсан.

Шийдэл: §4.2.2-т тайлбарласан 4 шатлалт fallback механизмыг хэрэгжүүлснээр систем гацахгүй ажиллах болсон. Зурган PDF-ийг цаашид OCR-оор дэмжих хэрэгтэй гэж дүгнэсэн.

4.10.4 AI хариулт удаашрах асуудал

Асуудал: Retriever-аас хэт олон текстийн хэсэг дамжуулах үед LLM хариулт үүсгэх хугацаа мэдэгдэхүйц өссөн. Офлайн орчинд CPU ашиглаж байсан тул энэ асуудал илүү мэд-

рэгдсэн.

Шийдэл: Зөвхөн хамгийн холбоотой Top-K chunk-ийг сонгож prompt-д оруулах байдлаар оновчлол хийсэн. Top-K-ийн өгөгдмөл утгыг 5 болгосон.

4.10.5 Дахин индексжүүлэх хугацаа их зарцуулах

Асуудал: Системийг дахин ажиллуулах бүрт бүх файлыг дахин embedding хийж байсан нь хугацаа их зарцуулж байв.

Шийдэл: §4.6-т тайлбарласан MD5 hash-д суурилсан кэш системийг нэмснээр дахин ажиллуулах хугацаа 614 секундээс 5 секунд болж буурсан.

5 Судалгааны үр дүн, дүгнэлт

5.1 Хайлтын туршилтын үр дүн

Системийн бодит гүйцэтгэлийг Python-ийн `time.perf_counter()` функцийг ашиглан хэмжсэн. Туршилтыг гурван өөр хэмжээний өгөгдөл дээр гүйцэтгэсэн.

Хүснэгт 5: Системийн бодит гүйцэтгэлийн хэмжилт (8 core CPU, 7.7GB RAM)

Файлын тоо	Chunk тоо	Индекс (сек)	Хайлт avg	AI хариулт avg	Embed хурд
464	4,135	65	0.018с	2.1с	63 chunk/c
1,000	12,500	195	0.022с	2.3с	63 chunk/c
4,000	60,862	614	0.035с	2.5с	63 chunk/C

4000 файл, 60,862 chunk-ийг 614 секундэд индексэлж чадсан бөгөөд зорилтот 700 секундэд багтсан. Хайлтыг дунджаар 0.035 секундэд гүйцэтгэсэн нь бодит хэрэглээнд ашиглах боломжтой хурдыг харуулж байна.

5.2 Гүйцэтгэлийн задаргаа (4000 файл)

Хүснэгт 6: Индексэлэлтийн алхам бүрийн хугацаа

Алхам	Хугацаа (сек)	Хувь
Файл уншилт (4000 файл)	50	8%
Chunking	5	1%
BM25 индекс	8	1%
Embedding (ONNX, 63/c)	540	88%
FAISS индекс	5	1%
Хадгалалт	6	1%
Нийт	614	100%

Embedding нь нийт хугацааны 88%-ийг эзэлж байгаа нь гол bottleneck юм. GPU ашиглавал энэ хугацааг 10–50 дахин багасгах боломжтой.

5.3 Оновчлолын үр дүн

Хүснэгт 7: Оновчлолын нөлөө

Оновчлол	Өмнө	Дараа
ONNX INT8 quantization	4 chunk/c	7 chunk/c
OpenVINO хурдасгал	7 chunk/c	63 chunk/c
Chunk size 600→900	90,000 chunk	60,862 chunk
xlsx/csv rows хязгаар	100с/файл	1с/файл
Embedding кэш	614с (дахин)	5с (дахин)

5.4 Дүгнэлт

Энэхүү судалгааны ажлаар бүрэн офлайн орчинд ажилладаг Hybrid RAG систем амжилттай хэрэгжүүлсэн. Системийн гол давуу талууд:

1. **Hybrid хайлт** — Вектор + BM25 + Keyword гурван аргыг хослуулснаар нэг аргад найдахаас илүү нарийвчлалтай үр дүн гардаг.
2. **Бүрэн офлайн** — Интернет холболт шаардахгүй, өгөгдлийн нууцлал бүрэн хангагдана.
3. **Олон формат** — 10 төрлийн файл дэмждэг (PDF-д 4 шатлалт fallback).
4. **Оновчлол** — ONNX INT8 + OpenVINO ашиглан GPU-гүй машин дээр 63 chunk/c хурдтай ажилладаг.
5. **Масштаблаг** — 4000+ файл, 60,000+ chunk-ийг 700 секундэд индексэлж чаддаг.
6. **Кэш систем** — Дахин индексэлэхэд зөвхөн өөрчлөгдсөн файлуудыг боловсруулна.

Цаашид хөгжүүлэх боломжууд:

- OCR нэмж зурган PDF-ийг дэмжих
- GPU хурдасгал (CUDA) бүрэн хэрэгжүүлэх
- Монгол хэлний тусгай embedding загвар сургах
- Олон хэрэглэгчийн горим нэмэх

Ном зүй

1. Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
2. Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data.
3. Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval.
4. Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019.
5. LangChain Documentation. (2024). *RetrievalQA Chain*. <https://docs.langchain.com/>
6. Gradio Documentation. (2024). *Building ML Demos*. <https://gradio.app/docs/>
7. Ollama Documentation. (2024). *Run LLMs Locally*. <https://ollama.com/>
8. Benedetti, M. (2024). *Evaluating RAG Systems with RAGAS Framework*. University of Padova.
9. Zheng, L. et al. (2025). *New Metrics for RAG Evaluation*. arXiv preprint.
10. Manrique, R. et al. (2024). *RAG Framework for Academic Papers*. Data Science Conference.
11. Haystack Documentation. (2024). *deepset Haystack NLP Framework*. <https://haystack.deepset.ai/>
12. PrivateGPT. (2024). *Interact with documents, 100% privately*. <https://privategpt.dev/>
13. ONNX Runtime. (2024). *Cross-platform ML Inference*. <https://onnxruntime.ai/>
14. APQC. (2023). *Knowledge Management Benchmark Report*. American Productivity & Quality Center.